

FitBuddy – AI Fitness Plan Generator Using Gemini Models

Project Description

FitBuddy is a web-based AI fitness application that creates personalized workout plans and nutrition tips based on a user’s fitness goals, such as *weight loss*, *muscle gain*, or *general wellness*. This application collects basic information such as *age*, *weight*, *fitness goal*, and *preferred workout intensity*.

Using Google Gemini AI, it generates a customized 7-day workout plan along with nutrition or recovery tips.

Users can also provide feedback to modify and improve their existing fitness plan.

The application provides a simple web interface as well as API endpoints, making it easy for users to create, view, and update their personalized fitness plans

System Architecture

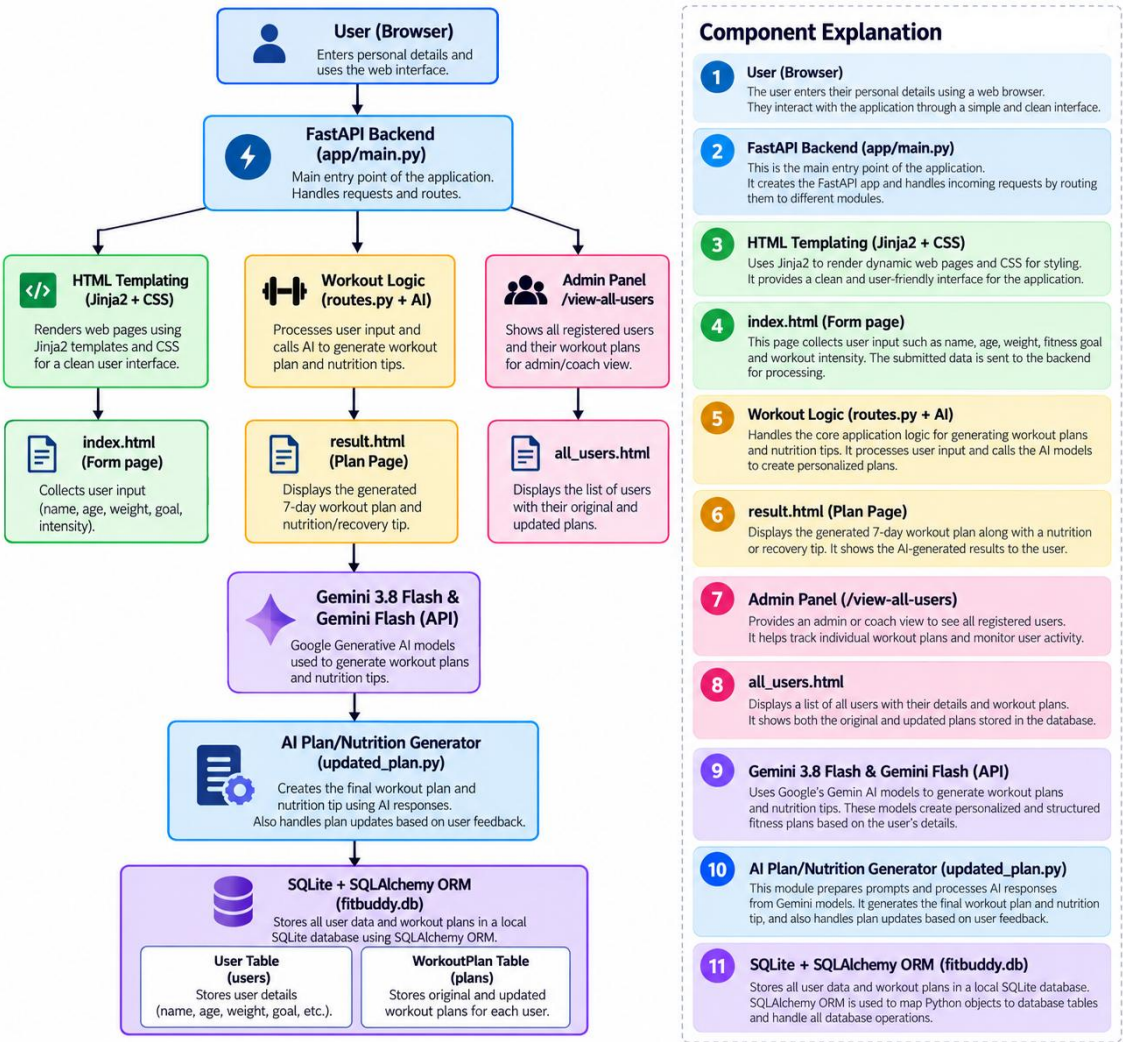


Table of Contents

Content	Page No
1. Executive Summary	3
2. Project Requirements	3
3. Complete Project Structure	4
4. Technology Stack	5
5. Application Startup Flow	6
6. main.py Detailed Explanation	6
7. Configuration Management	6
8. Pydantic Schemas	7
9. Database Architecture	7
10. Database Functions	8
11. FastAPI Routing Architecture	8
12. Request Processing	8
13. Workout Generation	9
14. Nutrition / Recovery Tip Generation	9
15. Feedback-Based Plan Update	9
16. Gemini Client	10
17. Frontend Architecture	10
18. CSS / UI Design	10
19. Complete Workout Generation Flow	13
20. Feedback Flow	14
21. Admin Workflow	14
22. REST API Documentation	14
23. Error Handling	15
24. Security	15
25. Installation and VS Code Setup	16
26. Running the Application	16
27. Database Access	16
28. Dependency Explanation	17
29. Source Code Walkthrough	17
30. Limitations	17
31. Future Enhancements	18
32. Troubleshooting Guide	18
33. Complete Project Flow Summary	18
34. Glossary	18
35. Conclusion	19
Appendix A – Complete Project File Structure	19
Appendix B – API Quick Reference	20
Appendix C – VS Code Setup and Command Cheat Sheet	20

1. Executive Summary

1.1 Short Executive Summary

FitBuddy is a FastAPI-based web and REST application that creates personalized seven-day workout plans and concise nutrition/recovery guidance. The current implementation validates user data with Pydantic, invokes Google Gemini through the google-genai SDK when real AI mode is enabled, persists user and plan data through SQLAlchemy into SQLite, renders browser pages with Jinja2, and supports feedback-driven plan revision.

1.2 Detailed Project Overview

The user enters a name, numeric user ID, age, weight, fitness goal and preferred intensity. The backend validates these values, generates a workout plan and nutrition/recovery tip, saves the profile and original plan, and renders the result page. A user can then submit feedback; the backend retrieves the original plan, asks the workout-update service to revise it, stores the revised version separately, and renders the updated result. An admin-style page lists all users and supports deletion. JSON endpoints expose the same core workout, feedback and user-list functionality.

The PDF baseline presents the same core product concept and a modular FastAPI/Jinja2/Gemini/SQLite architecture, including a 7-day plan, nutrition tip, feedback loop and admin dashboard. The current ZIP extends that baseline with a dedicated Gemini client, retry logic, JSON REST endpoints, mock AI mode, a shared base template, external CSS, automated tests, a health endpoint, and delete-user functionality.

Layer	Current role
Presentation	Jinja2 templates and CSS provide browser forms, results and admin UI.
Application/API	FastAPI exposes HTML routes, JSON routes, health check and OpenAPI/Swagger.
Validation	Pydantic UserInput and FeedbackRequest enforce ranges, lengths and allowed intensity values.
AI	Service modules construct prompts; gemini_client.py invokes Google GenAI and retries selected server errors.
Persistence	SQLAlchemy ORM maps users and workout_plans to SQLite.

2. Project Requirements

2.1 Functional Requirements Supported by the Current ZIP

- Collect user_id, username, age, weight, fitness goal and workout intensity.
- Validate input before business logic executes.
- Generate a complete 7-day workout plan, with mock and Gemini-backed execution paths.
- Generate a concise nutrition/recovery tip.
- Persist a user profile and one workout-plan record per user.
- Preserve original_plan while storing feedback revisions in updated_plan.
- Store the most recent feedback in last_feedback.
- Render a result page and accept feedback through HTML forms.
- List users and plans in an admin-style dashboard.
- Delete a user; the ORM relationship is configured with delete-orphan cascade for the related plan.
- Expose JSON APIs for workout generation, feedback update and user listing.
- Expose /health and FastAPI's generated /docs interface.

2.2 Non-Functional Characteristics

Area	Observed current behavior
Maintainability	Application is separated into config, schemas, database, routes, AI services, templates and CSS.
Modularity	Gemini invocation is centralized in gemini_client.py; prompts live in

	feature-specific service modules.
Reliability	Gemini server errors 500/502/503/504 are retried with exponential backoff and jitter.
Security	Secrets are intended to be stored in .env; .env is ignored by .gitignore; no authentication/authorization is implemented.
Scalability	SQLite and synchronous AI calls are suitable for a small/local project but are limiting for larger concurrent production workloads.
Usability	Responsive browser UI, HTML form constraints, feedback workflow and admin view are provided.
Persistence	SQLAlchemy persists users and one plan record per user in SQLite.
Error handling	Pydantic validation, 404s for missing user/plan, 502s around API AI failures, and HTML error rendering are partially implemented.

3. Complete Project Structure

The archive also contains a full .venv directory. It is intentionally omitted from the logical source tree below because it is generated environment content rather than project source.

```

FitBuddy-AI/
├── .env                # Real local configuration; contains secrets
├── .env.example        # Configuration template
├── fitbuddy.db         # SQLite database
├── pytest.ini
├── requirements.txt
├── .venv/              # Bundled virtual environment (generated content)
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── main.py
│   ├── routes.py
│   ├── schemas.py
│   ├── services/
│   │   ├── __init__.py
│   │   ├── gemini_client.py
│   │   ├── gemini_generator.py
│   │   ├── gemini_flash_generator.py
│   │   └── updated_plan.py
│   ├── static/
│   │   └── css/
│   │       └── styles.css
│   └── templates/
│       ├── all_users.html
│       ├── base.html
│       ├── index.html
│       └── result.html
└── tests/
    └── test_app.py

```

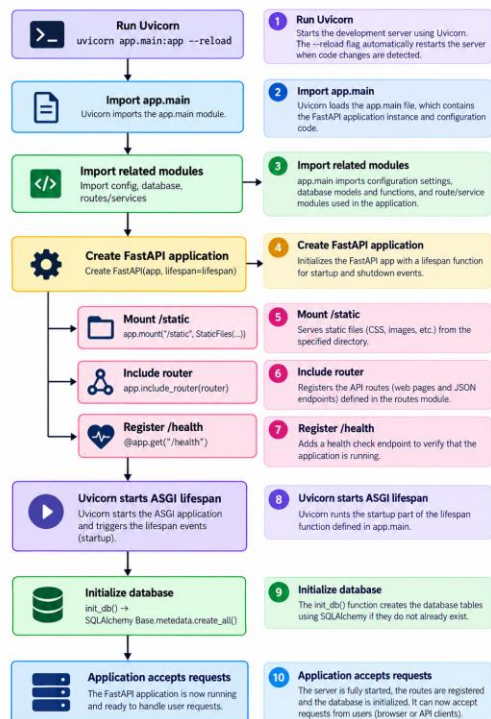
File	Purpose / important contents
app/main.py	Creates FastAPI app, lifespan DB initialization, static mount, router inclusion, /health.
app/routes.py	All HTML and JSON route handlers; coordinates validation, AI services, persistence and templates.
app/config.py	Pydantic Settings loaded from .env; app name, Gemini key/models, DB URL and mock flag.
app/schemas.py	UserInput and FeedbackRequest validation models.
app/database.py	SQLAlchemy engine/session, User/WorkoutPlan ORM models and CRUD-style helpers.
app/services/gemini_client.py	Google GenAI client, generate_content call, empty-response check, retry/backoff.
app/services/gemini_generator.py	Workout prompt plus deterministic mock workout.
app/services/gemini_flash_generator.py	Nutrition/recovery prompt plus mock tip.

app/services/updated_plan.py	Feedback-based plan-revision prompt plus mock revision.
templates/base.html	Shared page shell, navigation, stylesheet link and disclaimer footer.
templates/index.html	User input form.
templates/result.html	Profile, plan, tip and feedback form.
templates/all_users.html	Admin table with original/updated plan and delete form.
static/css/styles.css	Shared dark green responsive visual design.
tests/test_app.py	Single end-to-end test covering health, create workout, feedback and users API.
requirements.txt	Runtime and test dependencies.
pytest.ini	Adds project root to Python path and sets tests directory.

4. Technology Stack

Technology	Purpose	Where used
Python	Primary implementation language	All backend modules and tests
FastAPI	ASGI web/API framework	main.py, routes.py
Uvicorn	Development ASGI server	requirements and documented run command
Google GenAI SDK (google-genai)	Gemini API client	gemini_client.py
SQLAlchemy 2.x	ORM and database access	database.py
SQLite	Local relational database	DATABASE_URL and fitbuddy.db
Pydantic 2.x	Request validation	schemas.py; FastAPI JSON body validation
pydantic-settings	Environment-driven settings	config.py
Jinja2	Server-side HTML templates	routes.py and templates
HTML/CSS	Browser UI	templates and static/css/styles.css
python-multipart	Required for FastAPI Form(...) parsing	HTML POST forms
python-dotenv	Listed dependency; settings loading is performed by pydantic-settings env_file support	requirements.txt
Pytest	Automated testing	tests/test_app.py
HTTPX	Supporting dependency for FastAPI/Starlette TestClient stack	requirements/tests

5. Application Startup Flow



‘**uvicorn**’ is the ASGI server executable. ‘`app.main`’ identifies the Python module, and ‘`:app`’ identifies the FastAPI object in that module. ‘`--reload`’ watches source files and restarts the development process when code changes. On startup, the lifespan context calls `init_db()`, which creates missing ORM tables. Static files are mounted at `/static`, then the application serves HTML and JSON routes.

6. main.py Detailed Explanation

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield
```

The lifespan function is FastAPI's startup/shutdown context. Before yielding, it ensures the database tables exist. No explicit shutdown cleanup is required because database sessions are short-lived context-managed sessions.

```
app = FastAPI(
    title=settings.app_name,
    description="AI-powered personalized fitness plan generator ...",
    version="1.0.0",
    lifespan=lifespan,
)
```

The FastAPI object also supplies metadata used by OpenAPI and `/docs`. `StaticFiles` exposes `app/static` under `/static`.

‘`app.include_router(router)`’ imports all feature routes. ‘`/health`’ returns status, configured application name, and whether AI mode is ‘`gemini`’.

7. Configuration Management

`config.py` defines a `Settings` class backed by `pydantic-settings`. ‘`SettingsConfigDict(env_file='.env')`’ loads local environment values. ‘`get_settings()`’ is decorated with `lru_cache` so modules share one `Settings` instance per process.

Setting	Default in source	Purpose
<code>app_name</code>	FitBuddy – AI Fitness Plan Generator	OpenAPI title and health response
<code>gemini_api_key</code>	None	Credential passed to <code>genai.Client</code>
<code>gemini_workout_model</code>	<code>gemini-3.8-flash</code>	Configured workout/update model name
<code>gemini_tip_model</code>	<code>gemini-3.5-flash-lite</code>	Configured nutrition-tip model name
<code>database_url</code>	<code>sqlite:///fitbuddy.db</code>	SQLAlchemy database connection URL
<code>mock_ai</code>	False	Switches feature services to deterministic local responses

The real `.env` in the uploaded ZIP also contains `ADMIN_TOKEN` and `APP_NAME`. ‘`APP_NAME`’ is recognized automatically by the `Settings` field name. ‘`ADMIN_TOKEN`’ is not defined in `Settings` and, because ‘`extra='ignore'`’, is ignored by the current application. Its value is intentionally not exposed here.

```
.env
-> Settings(BaseSettings)
-> get_settings() [cached]
-> main.py / database.py / AI service modules
```

API keys should not be hardcoded because source files are routinely copied, committed, shared and logged. The supplied `.gitignore` excludes `.env`, although this particular ZIP contains a `.env` file; distribution packages should normally omit it.

8. Pydantic Schemas

Model	Field	Constraint
UserInput	<code>user_id</code>	integer > 0
UserInput	<code>username</code>	2–80 chars; whitespace normalized
UserInput	<code>age</code>	13–100 inclusive

UserInput	weight	greater than 20 and <= 400
UserInput	goal	3–200 chars; whitespace normalized
UserInput	intensity	literal: low, medium, high
FeedbackRequest	user_id	integer > 0
FeedbackRequest	feedback	3–1000 chars; leading/trailing whitespace stripped

The HTML workout route lowercases intensity before constructing UserInput. The JSON API relies on Pydantic directly and therefore expects one of the exact literal values. Pydantic prevents invalid values from reaching the AI and database layers.

9. Database Architecture

database.py creates an SQLAlchemy engine from DATABASE_URL. For SQLite only, `check\_same\_thread=False` is enabled because FastAPI can execute request handlers across threads. SessionLocal uses autoflush=False and expire_on_commit=False.

9.1 users table

Column	Type	Constraint	Description
id	INTEGER	Primary key	User-supplied user ID
username	VARCHAR(80)	NOT NULL	Display name
age	INTEGER	NOT NULL	Age
weight	FLOAT	NOT NULL	Weight in kg
goal	VARCHAR(200)	NOT NULL	Fitness goal
intensity	VARCHAR(20)	NOT NULL	low/medium/high
created_at	DATETIME	NOT NULL/default	UTC creation timestamp

9.2 workout_plans table

Column	Type	Constraint	Description
id	INTEGER	Primary key, autoincrement	Plan row identifier
user_id	INTEGER	FK users.id, UNIQUE, NOT NULL	One plan row per user
original_plan	TEXT	NOT NULL	Most recently generated original plan
updated_plan	TEXT	nullable	Latest feedback-refined plan
last_feedback	TEXT	nullable	Latest feedback text

```
erDiagram
    USERS ||--o| WORKOUT_PLANS : has
    USERS {
        int id PK
        string username
        int age
        float weight
        string goal
        string intensity
        datetime created_at
    }
    WORKOUT_PLANS {
        int id PK
        int user_id FK UK
        text original_plan
        text updated_plan
        text last_feedback
    }
```

The relationship is one-to-one (`uselist=False`) and configured with `cascade='all, delete-orphan'`. Deleting a loaded User through the ORM therefore deletes the associated WorkoutPlan object as part of the ORM unit of work.

10. Database Functions

Function	Purpose	Return / behavior
----------	---------	-------------------

init_db()	Create missing tables from ORM metadata	None
save_user(...)	Insert user or update existing user with same ID	Refreshed User
save_plan(user_id, plan)	Insert plan or replace original plan for existing user's plan; clears updated_plan and last_feedback	WorkoutPlan
update_plan(user_id, updated_text, feedback)	Store revised plan and latest feedback	WorkoutPlan or None
get_original_plan(user_id)	Retrieve original plan only	str or None
get_user(user_id)	Retrieve user with joined-loaded plan	User or None
get_all_users()	Retrieve users ordered by ID with joined-loaded plan	list[User]
delete_user(user_id)	Delete user; ORM cascade handles plan	bool

A feedback update does not overwrite original_plan. It writes to updated_plan, preserving the original and revised versions. A completely new generation for an existing user ID calls save_plan(), which replaces original_plan and explicitly resets updated_plan and last_feedback.

11. FastAPI Routing Architecture

Method	Endpoint	Purpose	Input	Output
GET	/	Render input form	Request	index.html
POST	/generate-workout	Generate/save plan and tip	HTML Form	result.html
POST	/submit-feedback	Revise existing plan	HTML Form user_id + feedback	result.html or HTTP error
GET	/view-all-users	Admin list	Request	all_users.html
POST	/delete-user/{user_id}	Delete user and redirect	Path user_id	303 -> /view-all-users
POST	/api/workouts	Generate/save plan and tip	UserInput JSON	JSON user + plan + tip
POST	/api/feedback	Revise existing plan	FeedbackRequest JSON	JSON user_id + updated_plan
GET	/api/users	List users/plans	None	JSON array
GET	/health	Health/mode	None	JSON status

12. Request Processing

```

Browser Form
-> FastAPI Form(...) parsing
-> UserInput / FeedbackRequest validation
-> AI service + database helpers
-> Jinja2 TemplateResponse
-> HTML response

JSON Client
-> FastAPI body parsing
-> Pydantic model validation
-> AI service + database helpers
-> dict/list
-> JSON response

```

The browser workout handler catches Pydantic ValidationError and re-renders index.html with HTTP 422. General workout-generation errors are rendered as HTTP 500 on the form. The feedback HTML handler explicitly converts Pydantic validation errors to HTTP 422 but otherwise allows raised HTTPException or other exceptions to propagate.

13. Workout Generation

`app/services/gemini\_generator.py` implements `generate\_workout\_gemini(age, weight, goal, intensity)`. In mock mode it returns `\_mock\_plan(goal, intensity)`, a fixed seven-day plan whose goal and intensity header are parameterized. In real mode it builds a prompt containing age, weight, goal and intensity.

- Requires exactly seven labeled days.
- Requests 5–10 minute warm-ups on workout days.
- Requests exercises with sets/repetitions or duration and sensible rest periods.
- Requests cooldown/stretching/mobility/recovery guidance.
- Requires at least one rest or active-recovery day.
- Asks Gemini to adapt difficulty to Low/Medium/High and align exercises with the stated goal.
- Adds safety boundaries: no diagnosis, no guaranteed outcome, and a short safety note.
- Requests plain text and explicitly rejects Markdown tables.

The feature then calls ``generate_text(settings.gemini_workout_model, prompt)``.

14. Nutrition / Recovery Tip Generation

``generate_nutrition_tip_with_flash(goal)`` reads settings and either returns a deterministic mock tip or constructs a short Gemini prompt. The real prompt requests 2–4 sentences, easy-to-understand advice, no diagnosis, no extreme dieting, no supplement prescriptions, no guaranteed results, and recognition that nutritional needs can vary. The output is rendered as ``nutrition_tip`` in `result.html`.

The source file retains the historical 'flash' name, but the actual model is controlled entirely by ``GEMINI_TIP_MODEL``; the current default configured in source is ``gemini-3.5-flash-lite``.

15. Feedback-Based Plan Update

```
sequenceDiagram
    participant U as User
    participant B as Browser
    participant R as FastAPI routes.py
    participant D as database.py
    participant A as updated_plan.py
    participant G as Gemini
    U->>B: Submit user_id + feedback
    B->>R: POST /submit-feedback
    R->>R: FeedbackRequest validation
    R->>D: get_user(user_id)
    R->>D: get_original_plan(user_id)
    R->>A: update_workout_plan(original, feedback)
    A->>G: generate_text(workout_model, prompt)
    G-->>A: Complete revised 7-day plan
    A-->>R: revised
    R->>D: update_plan(user_id, revised, feedback)
    R->>R: generate nutrition tip for user.goal
    R-->>B: result.html (updated=True)
```

The update prompt instructs Gemini to preserve the complete seven-day structure, change only what feedback requires, keep warm-up/main exercise/rest/cooldown elements, maintain sensible progression, avoid diagnosis and guaranteed results, and return the complete revised plan. The original database field remains unchanged; revised text is written to `updated_plan`.

16. Gemini Client

``gemini_client.py`` imports ``genai`` and ``errors`` from the ``google-genai`` package. ``get_client()`` is cached. It returns `None` in mock mode, rejects missing `GEMINI_API_KEY` in real mode, and otherwise constructs ``genai.Client(api_key=...)``.

```
response = client.models.generate_content(
    model=model,
    contents=prompt,
)
text = (response.text or "").strip()
```

`generate_text()` rejects empty responses. It retries Google GenAI `ServerError` only when the error code is 500, 502, 503 or 504. The delay uses exponential backoff plus random jitter, approximately 2, 4, 8 and 16 seconds before later attempts. After the fifth failed server attempt it raises a user-oriented `RuntimeError`. `ClientError` (4xx family such as authentication/quota/request problems) is re-raised without retry.

PDF baseline vs current code: the PDF describes `google-generativeai`, Gemini 1.5 Pro for workout/update and Gemini Flash for tips. The ZIP instead imports the newer `google-genai` package and makes model IDs configurable. The defaults in the source are `gemini-3.8-flash` and `gemini-3.5-flash-lite`. This document records those names exactly as supplied; it does not assert their external availability.

17. Frontend Architecture

Template	Current responsibility
base.html	Shared document shell, stylesheet, FitBuddy brand, Generate Plan/Admin/API Docs navigation, footer safety disclaimer.
index.html	Hero + form for username, user_id, age, weight, goal and intensity; displays server-side error.
result.html	Shows plan state, user profile, workout plan, nutrition/recovery tip and feedback form.
all_users.html	Loops through users; shows profile, original/updated plan; posts delete action with browser confirm dialog.

Jinja2 binds objects and values supplied by routes.py. result.html handles both Pydantic `UserInput` (`user.user_id`) and ORM `User` (`user.id`) through an `is defined` conditional. This allows the same template to serve both initial generation and feedback revision.

18. CSS / UI Design

The current ZIP centralizes styling in `app/static/css/styles.css`, unlike the PDF baseline which describes embedded CSS and a gym-themed background image. The current design uses CSS custom properties, a dark green background, gradient panels, an accent green, sticky translucent header, cards, grid layouts, styled forms/buttons, preformatted plan blocks, responsive rules and an admin table wrapper.

18.1 Project Screenshots / Application User Interface

Then arrange the screenshots in the same order as the actual user journey:

1. **FitBuddy Home Page / User Input Form** — Main page accepts basic inputs of the user (Name, User ID, Age, Weight (kg), Fitness Goal, Workout Intensity)

The screenshot shows the 'Build My Plan' form on the FitBuddy website. The form is titled 'Build My Plan' and includes fields for Name, User ID, Age, Weight (kg), Fitness Goal, and Workout Intensity. A 'Generate My Plan' button is at the bottom. The background features a large heading 'Your personalized 7-day workout plan' and a subheading 'AI-POWERED FITNESS PLANNING'. A disclaimer at the bottom states: 'FitBuddy provides general fitness information and is not a substitute for professional medical care.'

FitBuddy

Generate Plan Admin API Docs

AI-POWERED FITNESS PLANNING

Your personalized 7-day workout plan

Enter your details and FitBuddy will create a structured workout routine plus a goal-aligned nutrition or recovery tip.

Build My Plan

Name:

User ID: Age:

Weight (kg):

Fitness Goal:

Workout Intensity:

Generate My Plan

FitBuddy provides general fitness information and is not a substitute for professional medical care.

2. Generated Workout Plan — The generated 7-day plan along with Nutrition / Recovery Tip

The screenshot shows the 'Kavinaya P's Workout Plan' page. It displays the user's details (User ID, Age, Weight, Intensity, Fitness Goal) and the generated 7-day workout plan. The plan includes a warm-up, main workout, and a nutrition/recovery tip. The background features a large heading 'Kavinaya P's Workout Plan' and a subheading 'PERSONALIZED PLAN'. A 'Create New Plan' button is at the top right.

FitBuddy

Generate Plan Admin API Docs

PERSONALIZED PLAN

Kavinaya P's Workout Plan

Create New Plan

User ID: Age: Weight: Intensity:

Fitness Goal:

7-Day Workout Plan

Hello! I am FitBuddy, your fitness planning assistant. Below is your personalized 7-day workout plan designed for a medium intensity level to support your weight loss and overall fitness goals.

Day 1: Full-Body Resistance Circuit

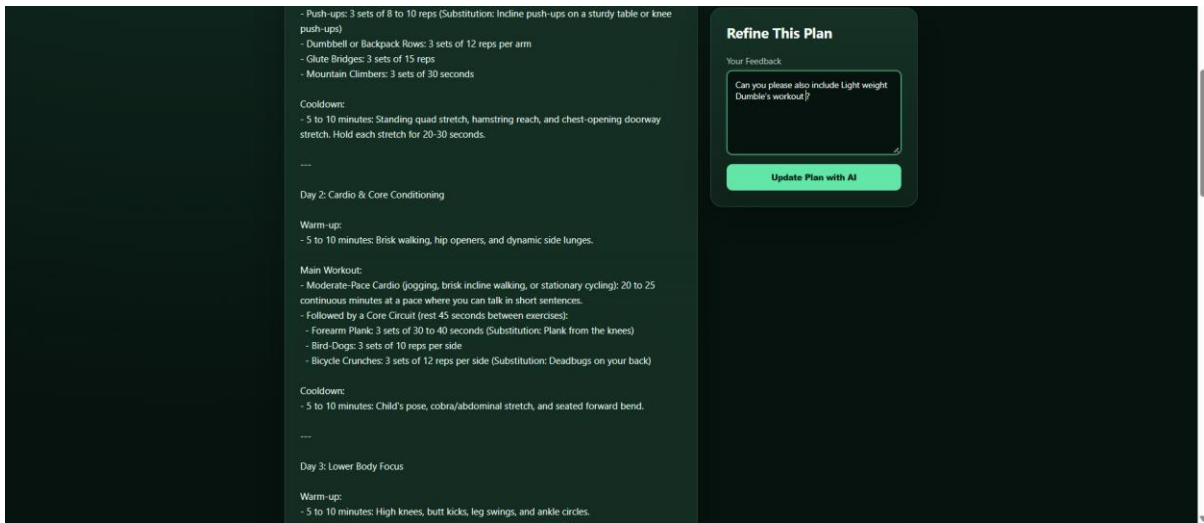
Warm-up:
- 5 to 10 minutes: Arm circles, torso twists, bodyweight squats, and light jogging in place.

Main Workout:
Perform each exercise with 60 seconds of rest between sets.
- Bodyweight Squats: 3 sets of 12 to 15 reps (Substitution: Sit-to-stand from a sturdy chair)

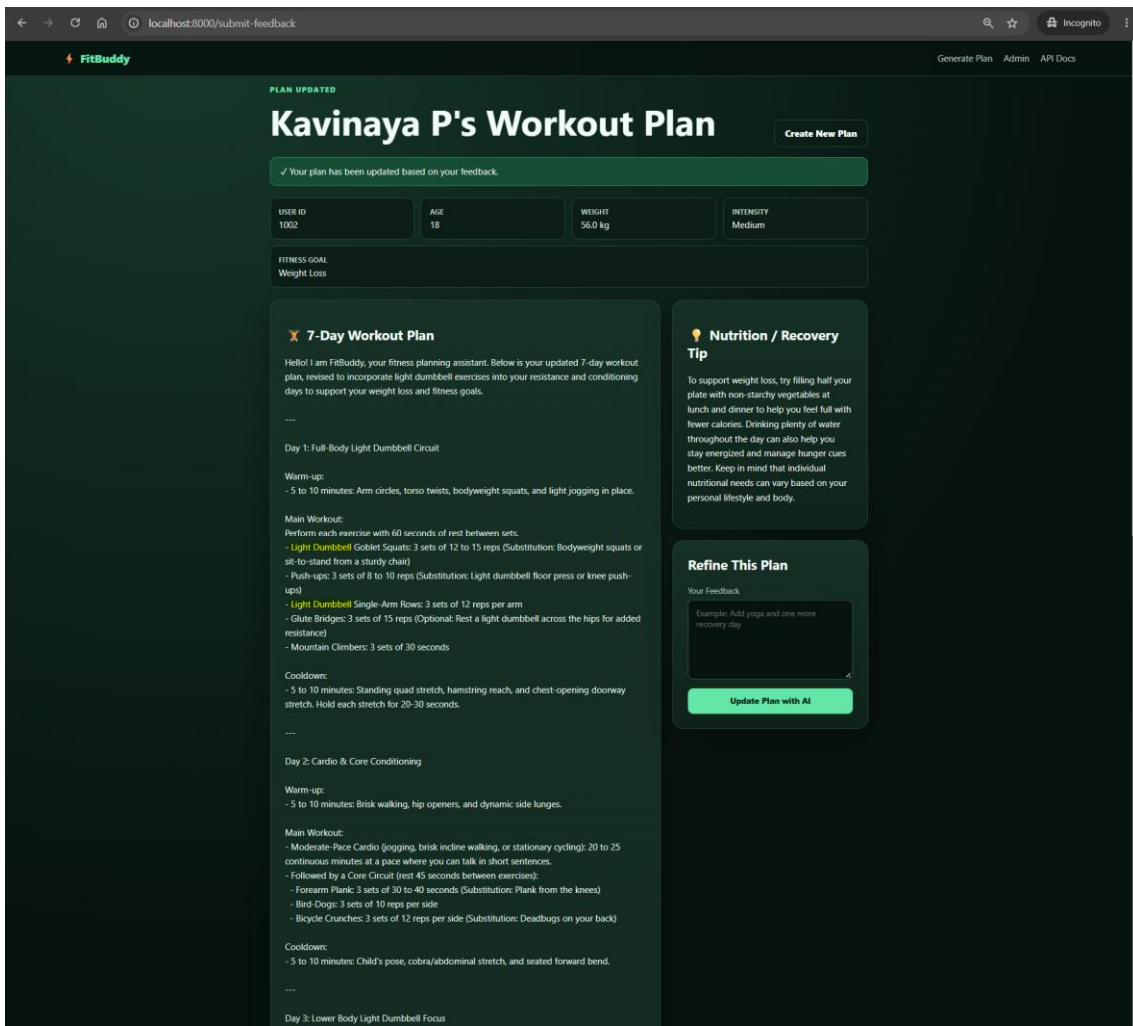
Nutrition / Recovery Tip

Hey there! For sustainable weight loss, try filling half your plate with non-starchy vegetables at lunch and dinner to help you feel full with fewer calories. Prioritizing lean proteins and staying well-hydrated throughout the day can also help manage cravings. Remember that individual nutritional needs can vary, so feel free to adjust these portions to what feels best for your body.

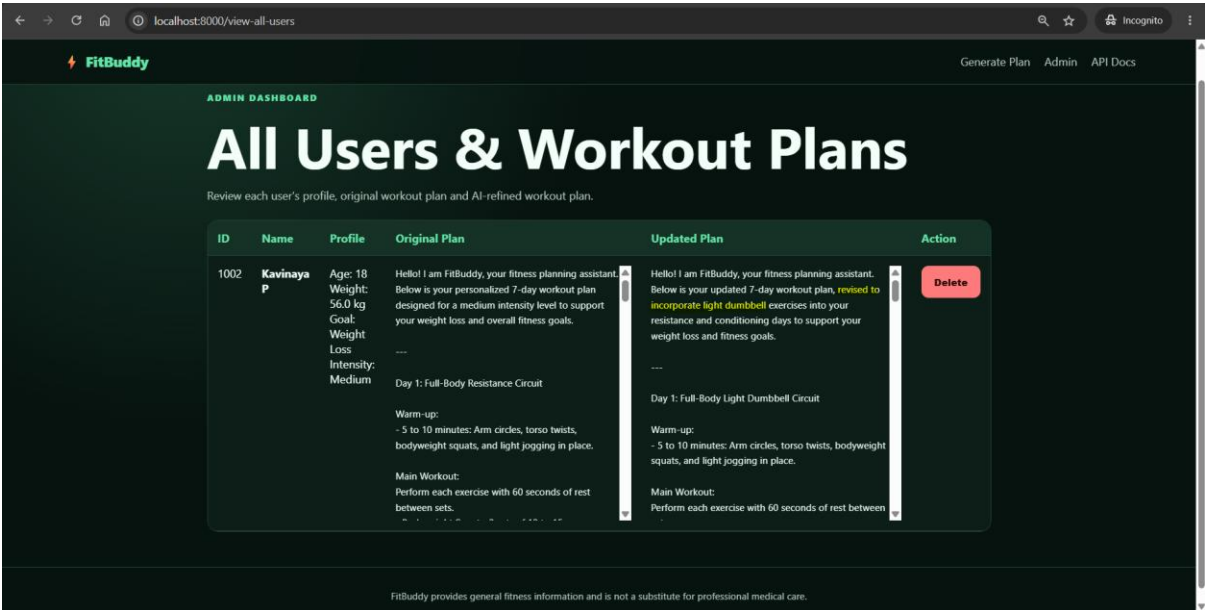
3. **Feedback Submission** — User also can request to include some of the personal plans along with the 7 – Day plan. The user has requested to include “**Light weight Dumbbells workout**” as well.



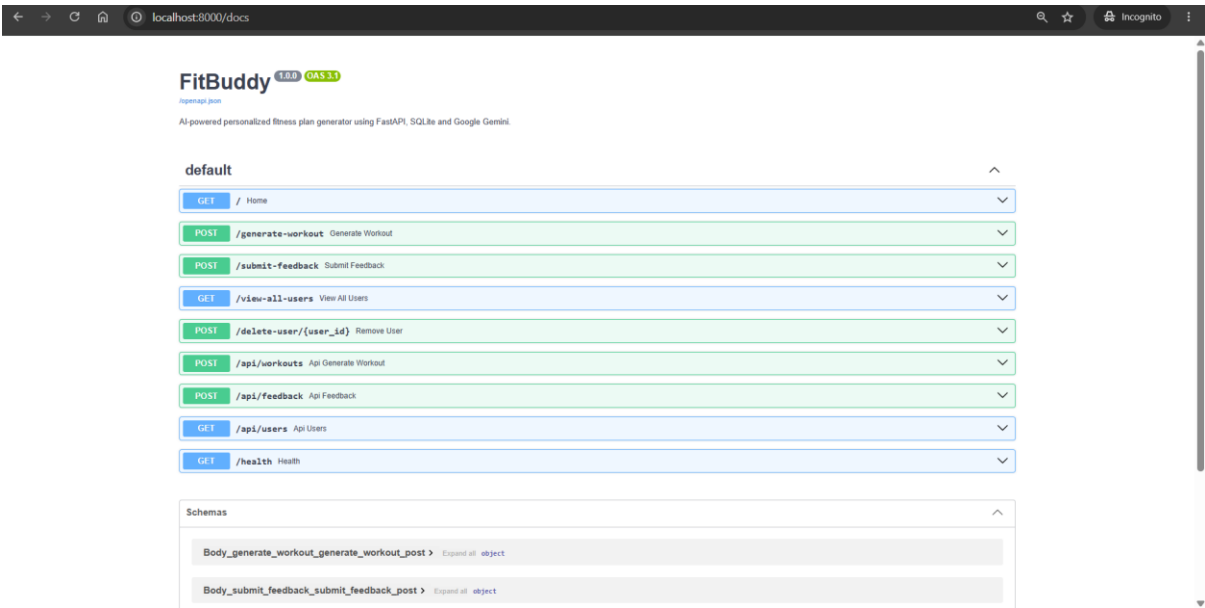
4. **Updated Workout Plan** – As we can see, **Light weight Dumbbells workout** plan also has been included.



5. **View All Users / Admin Dashboard** – As we can see, the original 7-day plan generated by the Gemini AI model and updated one as per the user request **Light weight Dumbbells workout**. The screen has every user’s Original vs. Updated plan as show below.



6. **API Documentation** – API Docs contains various pages’ request/responses.



19. Feedback Flow

The feedback path validates user_id and feedback, verifies that both user and original plan exist, revises the original plan through the update service, persists the revised plan and latest feedback, regenerates a nutrition/recovery tip using the user's stored goal, and reuses result.html with `updated=True`.

20. Admin Workflow

GET /view-all-users calls `get_all_users()`, which joined-loads each user's one-to-one plan. `all_users.html` displays ID, name, age, weight, goal, intensity, original plan and updated plan. A POST form targets `/delete-user/{id}` and asks for JavaScript confirmation.

There is no authentication or authorization protecting this admin route in the current source. Although the uploaded `.env` contains an `ADMIN_TOKEN` key, no Settings field or route logic consumes it. Therefore the dashboard and delete operation are open to any client that can reach the application.

21. REST API Documentation

22.1 POST /api/workouts

```
{
  "user_id": 101,
  "username": "Demo User",
  "age": 30,
  "weight": 70,
  "goal": "general wellness",
  "intensity": "medium"
}
```

Returns the validated user object, generated `workout_plan` and `nutrition_tip`. Pydantic validation failures are handled by FastAPI as 422. Exceptions inside the handler are converted to 502 with the exception string.

21.2 POST /api/feedback

```
{
  "user_id": 101,
  "feedback": "Add more mobility and one recovery session"
}
```

Returns `{user_id, updated_plan}`. Missing user or original plan returns 404. AI/update exceptions inside the try block return 502.

21.3 GET /api/users

Returns an array containing `user_id`, `username`, `age`, `weight`, `goal`, `intensity`, `original_plan` and `updated_plan`.

21.4 GET /health

Returns status, application name and AI mode. FastAPI also exposes `/docs` by default for Swagger UI and `/openapi.json` for the generated schema.

22. Error Handling

Condition	Current behavior
Invalid HTML workout data	Caught <code>ValidationError</code> ; <code>index.html</code> rendered with 422 and formatted error text.
Invalid JSON body	FastAPI/Pydantic returns standard 422 validation response.
Missing feedback user/plan	404 HTTPException.
Gemini empty response	<code>RuntimeError</code> from <code>generate_text()</code> .
Gemini 500/502/503/504	Retried up to 5 attempts with exponential backoff + jitter; final <code>RuntimeError</code> .
Gemini <code>ClientError</code> / 4xx	Re-raised without retry.
Workout JSON endpoint internal error	Converted to HTTP 502.
Feedback JSON endpoint AI/update error	Converted to HTTP 502.
HTML workout general error	<code>index.html</code> with status 500 and raw exception string.

HTML feedback non-validation AI error

No general catch in submit_feedback; framework-level 500 is possible.

Recommended production improvements: structured logging, stable user-safe error messages instead of raw exception strings, request correlation IDs, explicit database exception handling/rollback strategy, rate limiting, and centralized exception handlers.

23. Security

23.1 Already Implemented

- .env is listed in .gitignore.
- API key is read from settings rather than hardcoded in Python source.
- Pydantic validates user input lengths/ranges and restricts intensity.
- Jinja2 auto-escaping applies to normal template expressions.
- Basic fitness-safety language is included in prompts and the footer.

23.2 Current Limitations

- No authentication or authorization for the admin page, user listing, delete operation, or APIs.
- User IDs are client supplied and can overwrite an existing user's profile and plan.
- The uploaded distribution ZIP contains a real .env file despite .gitignore; secrets should not be distributed this way.
- ADMIN_TOKEN is present in the uploaded .env but unused by the code.
- No CSRF protection is visible for browser POST forms.
- No rate limiting is implemented.
- SQLite database is an unencrypted local file by default.
- Raw exception text can be returned/rendered in some error paths.

23.3 Production Recommendations

Add authentication, role-based authorization, CSRF defenses, secret-manager integration, rate limiting, HTTPS at the reverse proxy, structured audit logging, server-generated identities, least-privilege deployment, database backup/encryption controls as required, and a production database such as PostgreSQL when concurrency/durability requirements justify it.

24. Installation and VS Code Setup

1. Install a supported Python 3 release and VS Code.
2. Open the FitBuddy-AI folder in VS Code.
3. Open a PowerShell terminal in the project root.
4. Create a clean virtual environment: ``python -m venv .venv``.
5. Activate it: ``.venv\Scripts\Activate.ps1``.
6. Upgrade pip: ``python -m pip install --upgrade pip``.
7. Install dependencies: ``pip install -r requirements.txt``.
8. Copy ``.env.example`` to ``.env`` and set the required values. Never commit or share the real API key.
9. Start the app: ``uvicorn app.main:app --reload``.
10. Open the application and /docs in the browser.

25. Running the Application

```
uvicorn app.main:app --reload
```

Token	Meaning
uvicorn	Starts the ASGI server.
app.main	Imports module app/main.py.

:app	Uses the FastAPI object named app.
--reload	Development-only source watcher that restarts on code changes.
URL	Purpose
http://127.0.0.1:8000/	Browser application
http://127.0.0.1:8000/view-all-users	Admin-style user/plan dashboard
http://127.0.0.1:8000/docs	Swagger UI
http://127.0.0.1:8000/health	Health/mode JSON
http://127.0.0.1:8000/api/users	User/plan JSON list

26. Database Access

The configured SQLite file is `fitbuddy.db` when the default DATABASE_URL is used. The uploaded database contains the two ORM tables `users` and `workout\_plans`; at documentation time it contained one user row and one workout-plan row. No personal row values are reproduced here.

```
-- SQLite CLI
.tables
.schema users
.schema workout_plans

SELECT * FROM users;
SELECT * FROM workout_plans;

SELECT
    u.id,
    u.username,
    u.goal,
    u.intensity,
    p.original_plan,
    p.updated_plan,
    p.last_feedback
FROM users AS u
LEFT JOIN workout_plans AS p
    ON p.user_id = u.id
ORDER BY u.id;
```

In VS Code, a SQLite viewer extension can open fitbuddy.db directly. In Python, use the standard `sqlite3` module for inspection. The application also exposes the browser admin page and GET `/api/users`, though those should be protected before production use.

27. Dependency Explanation

Package	Role	Direct/supporting
fastapi	Web/API framework	Direct
uvicorn[standard]	ASGI development server and extras	Direct runtime
jinja2	HTML template engine	Direct
sqlalchemy	ORM/database toolkit	Direct
python-multipart	Parses multipart/form form submissions	Supporting FastAPI forms
python-dotenv	Environment-file utility; listed but not explicitly imported in source	Supporting/listed
google-gemai	Google Gemini SDK	Direct
pydantic	Data validation/models	Direct
pydantic-settings	Settings/env loading	Direct
httpx	HTTP client used by TestClient ecosystem; not explicitly imported by project source	Supporting test stack
pytest	Test runner	Direct test dependency

28. Source Code Walkthrough

11. Start with requirements.txt and .env.example to understand runtime dependencies and configuration.
12. Read config.py to see how environment values become typed settings.
13. Read main.py to see application construction, startup DB creation, static mounting and route inclusion.
14. Read schemas.py to understand accepted user/feedback data.
15. Read database.py to understand persistence and the one-to-one user-plan model.
16. Read gemini_client.py to understand real Gemini calls, retries and error handling.
17. Read gemini_generator.py, gemini_flash_generator.py and updated_plan.py to understand prompts and mock behavior.
18. Read routes.py to see orchestration across validation, AI, database and templates.
19. Read base/index/result/all_users templates and styles.css to understand the browser UI.
20. Read tests/test_app.py to see the expected end-to-end API flow.

29. Limitations

- SQLite is a single-file database and can become a concurrency/scaling constraint.
- Real AI operation depends on a valid Gemini API key, network access and the configured model being available to the account/project.
- AI calls are synchronous and use time.sleep during retries, tying up the request worker while waiting.
- No login, role-based authorization or protection of admin/delete/user-list endpoints.
- User IDs are supplied by clients and can update/replace existing records.
- Only one plan record exists per user; generating a new plan replaces the previous original and clears the previous update history.
- Only the latest feedback is retained; there is no plan/feedback history table.
- No migrations framework such as Alembic is present.
- No structured logging, monitoring, metrics, tracing or production deployment configuration is present.
- Fitness guidance is AI-generated general information; prompts include safety language but there is no clinical screening or professional review workflow.

30. Future Enhancements

The following are recommendations, not current features: add authentication and RBAC; server-generated IDs; PostgreSQL for larger deployments; Alembic migrations; asynchronous/background AI work where appropriate; structured retries with Retry-After handling; centralized logging/monitoring; Docker and CI/CD; progress and exercise history; versioned workout/feedback history; protected coach/admin roles; stronger health-safety intake and disclaimers; analytics; and, if privacy requirements demand it, a separately designed local/private LLM or RAG architecture.

31. Troubleshooting Guide

Problem	Practical check / action
ModuleNotFoundError: app	Run commands from the FitBuddy-AI project root; ensure pytest.ini is present.
Virtual environment issues	Delete/recreate .venv locally; do not reuse a .venv copied from another OS.
Missing package / google.genai import error	Activate venv and run `pip install -r requirements.txt`; verify `pip show google-genai`.
Missing Gemini key	Set GEMINI_API_KEY in .env, or use MOCK_AI=true for local mock operation.
Gemini 503	Current client retries selected server errors automatically; after final failure, retry later.
Gemini 429/quota	ClientError is not retried by current code; inspect account quota/billing/request rate and retry appropriately.
database is locked	Stop competing writers, close SQLite viewers holding transactions,

	then retry; consider production DB for higher concurrency.
Port 8000 in use	Run `uvicorn app.main:app --reload --port 8001` or stop the process using 8000.
Template not found	Run from project root and verify app/templates files are present.
Static CSS not loading	Verify app/static/css/styles.css and the `/static` mount in main.py.
Wrong working directory	Ensure terminal path is the folder containing requirements.txt, pytest.ini and app/.

32. Complete Project Flow Summary

User Input

- > FastAPI Form/JSON route
- > Pydantic Validation
- > Workout Generator
- > Gemini Client/API (or Mock)
- > Nutrition Tip Generator
- > Gemini Client/API (or Mock)
- > save_user + save_plan
- > SQLAlchemy -> SQLite
- > Jinja2 result page / JSON response
- > User Feedback
- > get_user + get_original_plan
- > Updated Plan Generator -> Gemini (or Mock)
- > update_plan(updated_plan + last_feedback)
- > Updated result / JSON response

33. Glossary

Term	Beginner-friendly meaning
FastAPI	Python framework for building web applications and APIs.
ASGI	Python standard interface used by asynchronous-capable web servers and frameworks.
Uvicorn	ASGI server that runs the FastAPI application.
REST API	HTTP endpoints that exchange structured data, usually JSON.
Gemini	Google's generative AI model family accessed here through an SDK/API.
Generative AI	AI that generates new content such as text from instructions/context.
Prompt	Text instructions/context sent to a generative model.
Pydantic	Python validation library used to define and validate typed input models.
ORM	Object-Relational Mapper; maps Python objects/classes to relational database tables.
SQLAlchemy	Python database toolkit/ORM used by FitBuddy.
SQLite	Embedded relational database stored in a local file.
Jinja2	Template engine that combines HTML with dynamic server data.
Environment variable	Configuration value supplied outside source code.
API key	Credential used to authorize API access.
CRUD	Create, Read, Update and Delete data operations.
HTTP status code	Numeric response result such as 200, 404, 422 or 502.

34. Conclusion

The current FitBuddy project is a compact, modular FastAPI application that combines browser pages, JSON APIs, validated user input, AI-generated fitness content and SQLite persistence. FastAPI coordinates routing and OpenAPI documentation; Pydantic validates requests; service modules construct workout, nutrition and feedback prompts; a shared Google GenAI client handles model calls and selected transient-server retries; SQLAlchemy persists the user and original/revised plan; and Jinja2 presents the result in a responsive UI.

Appendix A – Complete Project File Structure

Logical project source tree (generated .venv contents omitted):

```
FitBuddy-AI/
├── .env
├── .env.example
├── .gitignore
├── fitbuddy.db
├── pytest.ini
├── requirements.txt
├── .venv/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── main.py
│   ├── routes.py
│   ├── schemas.py
│   ├── services/
│   │   ├── __init__.py
│   │   ├── gemini_client.py
│   │   ├── gemini_flash_generator.py
│   │   ├── gemini_generator.py
│   │   └── updated_plan.py
│   ├── static/css/styles.css
│   └── templates/
│       ├── all_users.html
│       ├── base.html
│       ├── index.html
│       └── result.html
└── tests/test_app.py
```

Appendix B – API Quick Reference

Method	Path	Input	Primary success output
GET	/	—	HTML form
POST	/generate-workout	Form profile	HTML result
POST	/submit-feedback	Form user_id, feedback	HTML updated result
GET	/view-all-users	—	HTML admin table
POST	/delete-user/{user_id}	Path ID	303 redirect
POST	/api/workouts	UserInput JSON	user + workout_plan + nutrition_tip
POST	/api/feedback	FeedbackRequest JSON	user_id + updated_plan
GET	/api/users	—	JSON user/plan list
GET	/health	—	status + application + ai_mode
GET	/docs	—	Swagger UI

Appendix C – VS Code Setup and Command Cheat Sheet

```
# Create and activate environment
python -m venv .venv
.\venv\Scripts\Activate.ps1

# Install
python -m pip install --upgrade pip
pip install -r requirements.txt

# Create local config
Copy-Item .env.example .env
```

```
# Test
pytest -q

# Run
uvicorn app.main:app --reload

# Optional alternate port
uvicorn app.main:app --reload --port 8001
```

For a real Gemini run, place the API key in .env and set MOCK_AI=false. For local/offline functional testing, set MOCK_AI=true. Do not distribute the real .env file.